

Java 12 — Switch Expressions (Preview)

Aspect	Details
Java Version	Java 12
Year	2019
Feature	Switch expressions (Preview)
UniqueClassName	SwitchExpressions
Purpose	Allows switch to produce a value, making conditional value selection more concise and reducing traditional switch boilerplate.
Status in Java 12	Preview feature — required preview-feature compilation/runtime options.
Basic syntax	<pre>int result = switch (day) { case MONDAY -> 1; case TUESDAY -> 2; default -> 0; };</pre>

1. Advantages and Disadvantages

Category	Advantages	Disadvantages
Return values	switch can directly return/produce a value.	Developers unfamiliar with expression-oriented programming may initially find it unusual.
Boilerplate	Eliminates temporary variables and repetitive assignments.	Java 12 implementation was still a preview , so syntax/API details were subject to change.
Fall-through	Arrow (->) cases avoid accidental fall-through.	Traditional : syntax can still be confusing when mixed with newer syntax.
Readability	Simple value mappings become compact and easy to understand.	Very large or complicated switches can still become difficult to read.
Safety	Exhaustiveness can be enforced for suitable switch expressions, reducing missing-case problems.	Exhaustiveness rules can require a default depending on the switched type/cases.
Control flow	yield allows a multi-statement case to produce a	yield adds another keyword/concept to learn.

	value.	
Maintenance	Centralizes mapping logic and reduces mutable local variables.	Poorly designed expressions can become overly dense.
Modern Java style	Encourages expression-oriented code.	Codebases restricted to older Java versions cannot use it.
Performance	Can produce clean bytecode and gives the compiler/JVM opportunities for optimization.	It should not be assumed to be faster than an equivalent traditional switch; performance depends on the actual code and JVM.

2. When to Use vs. When Not to Use

Situation	Use Switch Expressions?	Reason	Example
Simple value mapping	✓ Use	Excellent fit for mapping one value to another.	status -> HTTP status code
Enum-to-value mapping	✓ Use	Very readable and naturally exhaustive.	OrderStatus -> display message
Command/type dispatch	✓ Use	Concise selection of behavior/value.	PaymentType -> fee
HTTP/status mapping	✓ Use	Removes repetitive assignments.	Status -> response code
Multiple statements required	✓ Use with yield	Allows a case to calculate a value.	Calculate discount based on category
Complex business workflow	□ Usually avoid	A large switch expression can hide complicated business logic.	50-case pricing engine
Side-effect-heavy logic	□ Usually avoid	Expressions are clearest when primarily producing values.	Multiple database writes in cases
Simple boolean condition	✗ Don't use	if/else is generally clearer.	age >= 18

Two alternatives	✗ Don't use	if/else is simpler.	premium ? 20 : 10
Legacy Java 8/11 application	✗ Don't use	Java 12 preview syntax isn't available without upgrading/enabling preview.	Java 11 production application
Java 12 production system requiring maximum stability	☐ Consider carefully	It was a preview feature.	Mission-critical application
Repeated complex behavior	☐ Usually avoid	Strategy/polymorphism may be more maintainable.	Different payment processors

3. Traditional switch vs. Switch Expression

Traditional switch	Switch expression
Primarily a statement	Produces a value
Often requires a temporary variable	Value can be assigned directly
Commonly uses break	Arrow cases eliminate normal fall-through
More verbose	More concise
Fall-through can happen accidentally	-> cases don't fall through
break exits a case	yield returns a value from a block
Good for imperative workflows	Excellent for value selection

Traditional approach:

```
int days;

switch (month) {
    case 2:
        days = 28;
        break;
    case 4:
    case 6:
```

```

    case 9:
    case 11:
        days = 30;
        break;
    default:
        days = 31;
}

```

Switch expression:

```

int days = switch (month) {
    case 2 -> 28;
    case 4, 6, 9, 11 -> 30;
    default -> 31;
};

```

The second version expresses the intent more directly:

"Choose a number of days based on the month."

4. Real-Time Use Cases

Real-Time Use Case	Example	Why Switch Expression Fits
Order status	PENDING -> "Awaiting payment"	Simple enum-to-message mapping
HTTP response	NOT_FOUND -> 404	Direct status-to-code conversion
Payment fee	CREDIT_CARD -> 2.5	Clear value selection
User roles	ADMIN -> "FULL_ACCESS"	Role-to-permission mapping
Shipping method	EXPRESS -> 99, STANDARD -> 49	Direct pricing calculation
Product category	ELECTRONICS -> 18	Category-to-tax-rate mapping
Log severity	ERROR -> 3, WARN -> 2	Severity-to-priority conversion
File type	PDF -> "application/pdf"	Type-to-MIME mapping
Game scoring	GOLD -> 100, SILVER -> 50	Simple scoring rules
API operation	CREATE -> "POST", UPDATE -> "PUT"	Operation-to-HTTP-method mapping

5. Real-Time Example — Order Status

```
enum OrderStatus {  
    CREATED, PAID, SHIPPED, DELIVERED, CANCELLED  
}  
  
String message = switch (status) {  
    case CREATED    -> "Order created";  
    case PAID       -> "Payment received";  
    case SHIPPED    -> "Order shipped";  
    case DELIVERED  -> "Order delivered";  
    case CANCELLED  -> "Order cancelled";  
};
```

This is particularly useful because the switch **produces the String directly**.

6. Real-Time Example — Using yield

For more complicated case logic:

```
String category = "ELECTRONICS";  
  
double discount = switch (category) {  
    case "ELECTRONICS" -> {  
        double baseDiscount = 10.0;  
        double seasonalBonus = 5.0;  
        yield baseDiscount + seasonalBonus;  
    }  
    case "CLOTHING" -> 15.0;  
    case "BOOKS" -> 8.0;  
    default -> 0.0;  
};
```

Here, yield supplies the value produced by the block.

7. Alternate Features

Alternative	Java Version	Suitable When	Compared with Switch Expressions
Traditional switch statement	Java 1.0+	You need imperative control flow or must	More verbose but broadly compatible.

		support older Java.	
if-else	Java 1.0+	Conditions involve ranges, complex boolean expressions, or only a few alternatives.	Often clearer for non-discrete conditions.
Polymorphism	Java 1.0+	Different types require substantially different behavior.	More extensible than a huge switch.
Strategy Pattern	Any	Business rules/algorithms vary and should be independently replaceable.	Better for complex, growing decision logic.
Map<K,V> lookup	Java 1.2+	The switch is simply a static key-to-value lookup.	Can be more data-driven and configurable.
Enum methods	Java 5+	Behavior naturally belongs to the enum itself.	Can eliminate the external switch completely.
Ternary operator	Java 1.0+	Only two simple alternatives exist.	More concise for binary choices.

8. Decision Guide

Requirement	Best Choice
2 simple alternatives	if/else or ternary
Several discrete values → one value	Switch expression
Enum → value/message	Switch expression
Complex boolean/range conditions	if/else
Simple static lookup table	Map
Different behavior for different object types	Polymorphism/Strategy

Behavior belongs naturally to an enum	Enum methods
Complex multi-step business rules	Strategy/Polymorphism
Java 11 compatibility required	Traditional switch
Java 12 and willing to use preview features	Switch expression (preview)

Key Takeaway

Switch expressions in Java 12 are best viewed as a cleaner way to say "select a value based on this discrete input." They are particularly strong for **enum/value mappings, status conversions, pricing rules, HTTP mappings, and similar decision tables.**

One important historical point: **Java 12's switch expressions were preview-only.** The feature subsequently evolved and became a permanent Java feature in later releases, so for modern Java development you should generally use the finalized switch-expression syntax rather than treating the Java 12 preview as the target.